STL笔记

Day20

一、标准模板库STL

六大组件

1、容器 用来存数据的，也就是数据结构

- 序列式容器 vector
- 关联式容器 set、map
- 无序关联式容器 unordered_set、unordered_map

2、迭代器 看成是指针，泛型指针，具备指针的功能。访问容器中的元素的。

3、算法 操作容器中的元素。迭代器可以看成是容器与算法之间的桥梁。

4、适配器 做适配操作

- 容器适配器 stack queue
- 迭代器适配器
- 函数适配器

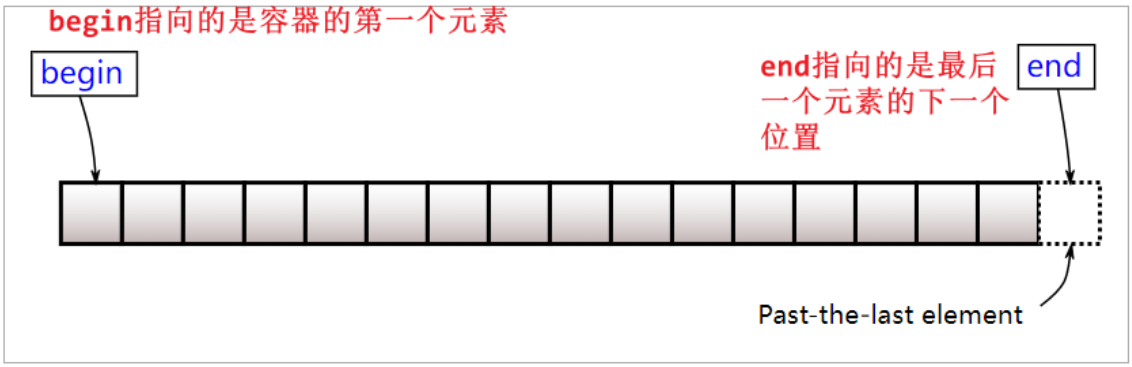
5、函数对象（仿函数） 定制化操作。

6、空间配置器 进行空间的申请与释放的。（用法 + 原理 + 源代码 ）

程序 = 数据结构（容器） + 算法。

二、序列式容器

```
//.bashrc的最后一行
export PS1='\[\e[36;47m\]\u\[\e[33;47m\]@\[\e[36;47m\]\h\[\e[35;47m\]:\[\e[31;47m\]\W\[\e[00m\]\$ '

//.vimrc下面，添加这么一行(多标签之间的切换)
nmap <Tab> :bn<cr>
```



1、初始化

三种容器的初始化方式是完全一样。都有五种初始化的方式。

```
    //初始化
    //1、无参对象
    /* vector<int> number; */
    /* vector<int> number2();//number2是函数名 */
    //2、count个value
    /* vector<int> number(10, 2); */
    //3、迭代器范围的形式
    /* int arr[10] = {1, 3, 5, 7, 9, 8, 6, 4, 2, 0}; */
    /* vector<int> number(arr, arr + 10);//[arr,arr + 10)左闭右开 */
    //4、拷贝构造或者移动构造
    /* vector<int> number2(number); */
    //5、大括号
    vector<int> number = {1, 3, 5, 7, 9, 2, 4, 6, 8};
```

2、遍历

vector与deque有四种遍历方式，但是list没有重载下标访问运算符。（不支持下标）

```
    //遍历
    //1、使用下标
    for(size_t idx = 0; idx != number.size(); ++idx)
    {
        cout << number[idx] << "  ";
    }
    cout << endl;

    //2、迭代器的形式
    //迭代器可以看成是指针，但是不完全等同于指针。泛型指针
    vector<int>::iterator it;
    for(it = number.begin(); it != number.end(); ++it)
    {
        cout << *it << "  ";
    }
    cout << endl;

    //3、初始化迭代器
    vector<int>::iterator it3 = number.begin();
    for(; it3 != number.end(); ++it3)
    {
        cout << *it3 << "  ";
    }
    cout << endl;

    //4、for与auto结合
    for(auto &elem : number)
    {
        cout << elem << "  ";
    }
    cout << endl;
```

注意：list不支持下标访问

```
    //遍历
    //1、使用下标
    for(size_t idx = 0; idx != number.size(); ++idx)
    {
        cout << number[idx] << "  ";    list是不支持下标访问
    }
    cout << endl;

```

```
-rw-rw-r--  1 wangdao wangdao     0 6月  10 08:52 .keep
wangdao@wangdao:CppBase_day20$
wangdao@wangdao:CppBase_day20$ g++ 3_list.cc
3_list.cc: In function 'void test()':
3_list.cc:28:23: error: no match for 'operator[]' (operand types are
 'std::__cxx11::list<int>' and 'size_t {aka long unsigned int}')
        cout << number[idx] << "  ";
                       ^
```

3、在尾部进行插入与删除

三种容器的在尾部的插入与删除是一致的。

```
    cout << endl << "在list尾部进行插入与删除" << endl;
    number.push_back(12);
    number.push_back(13);
    display(number);
    number.pop_back();
    display(number);
```

Day21

一、序列式容器

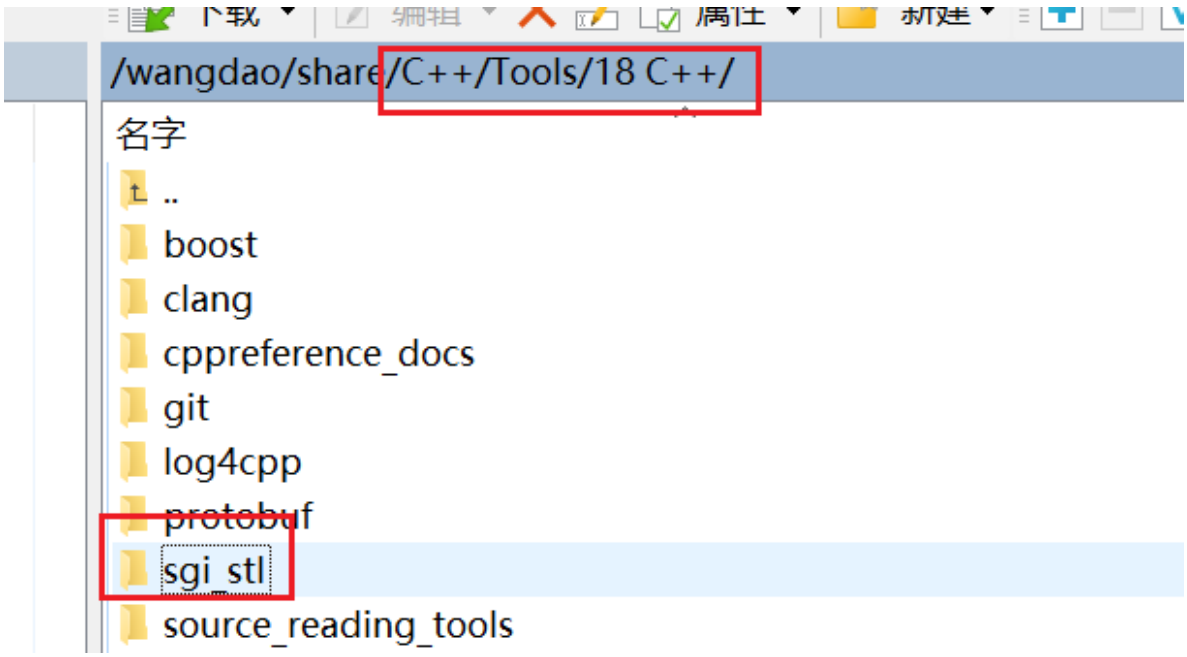
1、在头部进行插入与删除

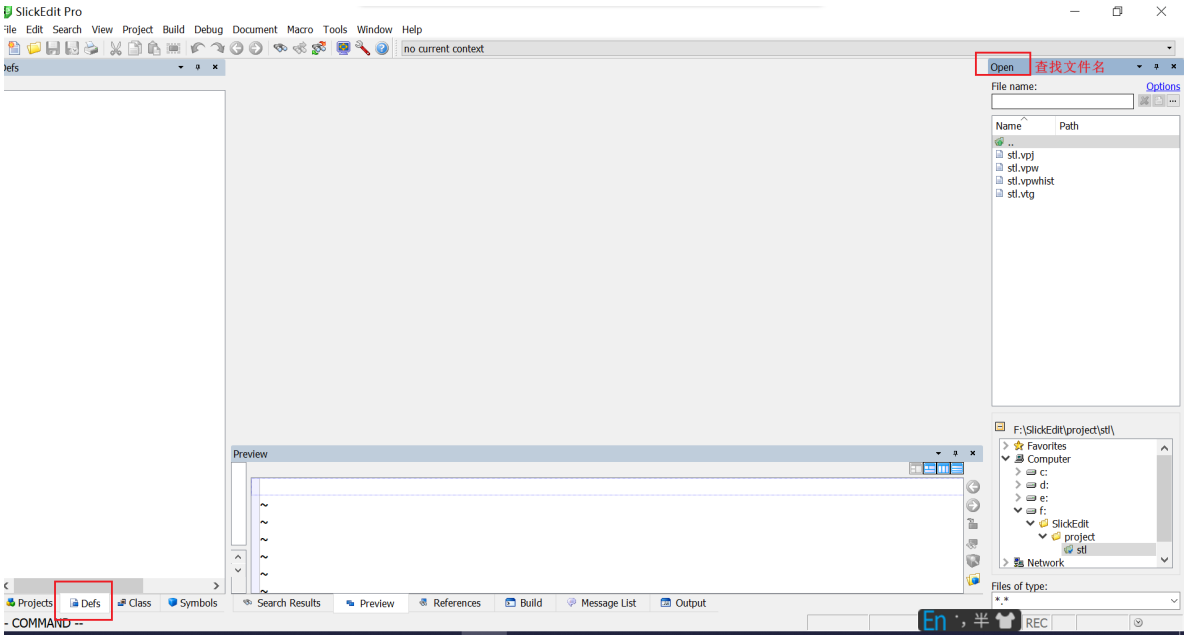
deque与list是支持在头部进行插入与删除的，但是vector不支持。

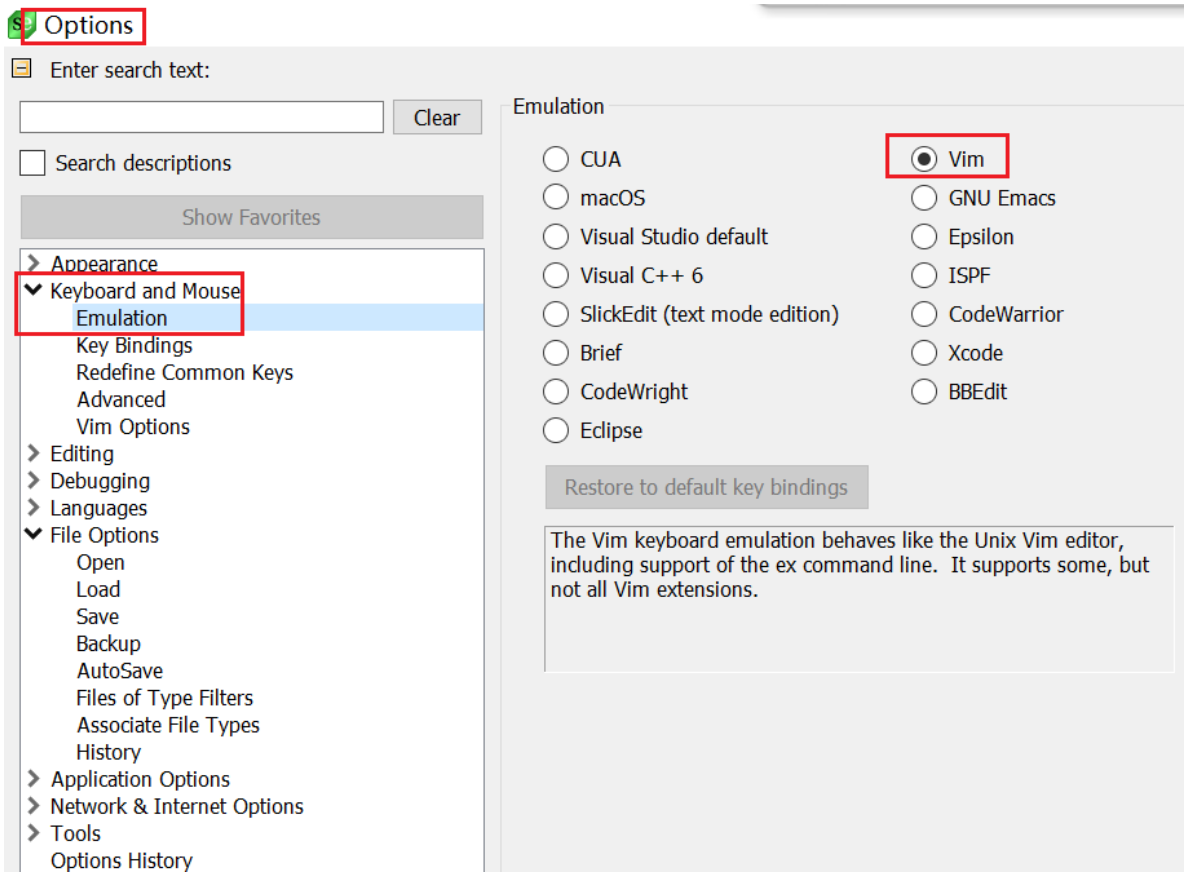
```

    //Q:为什么没有提供在头部进行插入与删除的方法？
    //A:vector中的元素是连续的，如果在头部插入一个元素或者删除
    //一个元素，会导致后面所有的元素都需要挪动位置，那么时间
    //复杂度O(N)
```

```
    cout << endl << "在deque头部进行插入与删除" << endl;
    number.push_front(100);
    number.push_front(200);
    display(number);
    number.pop_front();
    display(number);
```

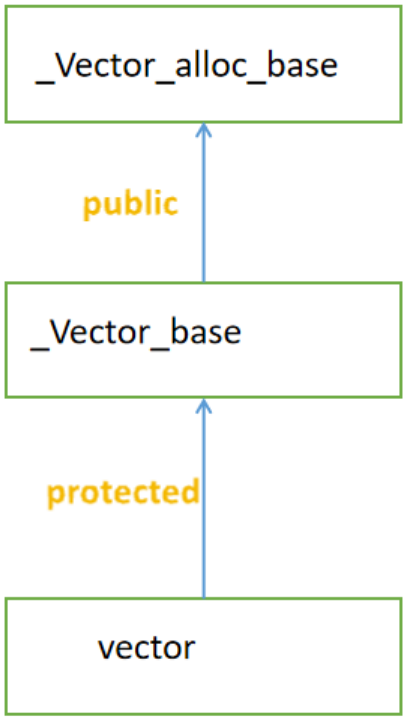
2、vector的源码




SlickEdit Pro
File Edit Search View Project Build Debug Document Macro Tools Window Help
no current context
Open
查找文件名
File name:
Options
Name
Path
stl.vpj
stl.vpw
stl.vpwhist
stl.vtg
F:\SlickEdit\project\stl\
Favorites
Computer
c:
d:
e:
f:
SlickEdit
project
stl
Network
Files of type:
.
Preview
Projects
Defs
Class
Symbols
Search Results
Preview
References
Build
Message List
Output
- COMMAND --
REC


Options
Enter search text:
Clear
Search descriptions
Show Favorites
Appearance
Keyboard and Mouse
Emulation
Key Bindings
Redefine Common Keys
Advanced
Vim Options
Editing
Debugging
Languages
File Options
Open
Load
Save
Backup
AutoSave
Files of Type Filters
Associate File Types
History
Application Options
Network & Internet Options
Tools
Options History
Emulation
CUA
macOS
Visual Studio default
Visual C++ 6
SlickEdit (text mode edition)
Brief
CodeWright
Eclipse
Vim
GNU Emacs
Epsilon
ISPF
CodeWarrior
Xcode
BBEdit
Restore to default key bindings
The Vim keyboard emulation behaves like the Unix Vim editor, including support of the ex command line. It supports some, but not all Vim extensions.

vector的继承图解



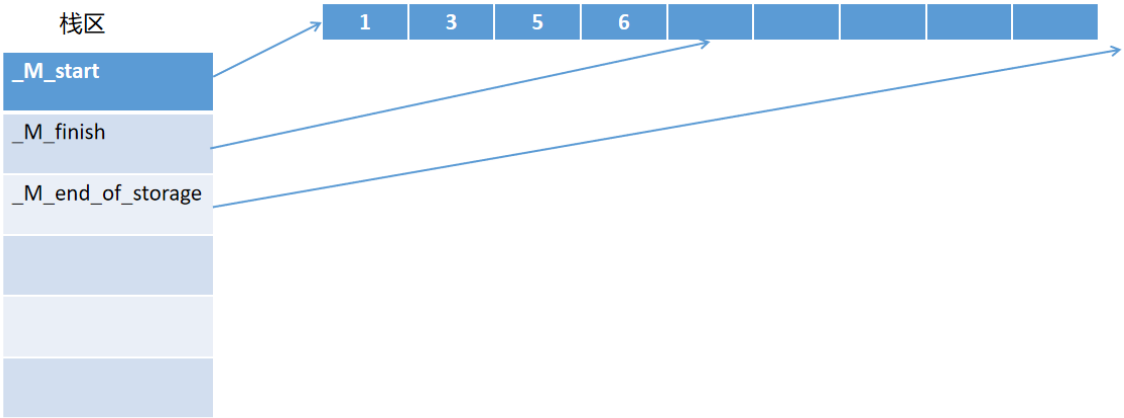

```
_Tp*    _M_start;
_Tp*    _M_finish;
_Tp*    _M_end_of_storage;
```

```
class vector
{
public:
  typedef _Tp value_type;
  typedef value_type* pointer;
  typedef const value_type* const_pointer;
  typedef value_type* iterator;
  typedef const value_type* const_iterator;
  typedef value_type& reference;
  typedef const value_type& const_reference;
  typedef size_t size_type;
  typedef ptrdiff_t difference_type;

   typedef typename _Base::allocator_type allocator_type;

};
```

```
//泛型编程的思想
//类型萃取
Container::size_type

Container::iterator

//size_t是无法符号的，如果是-1，就会转变为最大值
for(size_t idx = 9; idx > 0; idx -= 2)
{
    //9 7 5 3 1 -1
}

```

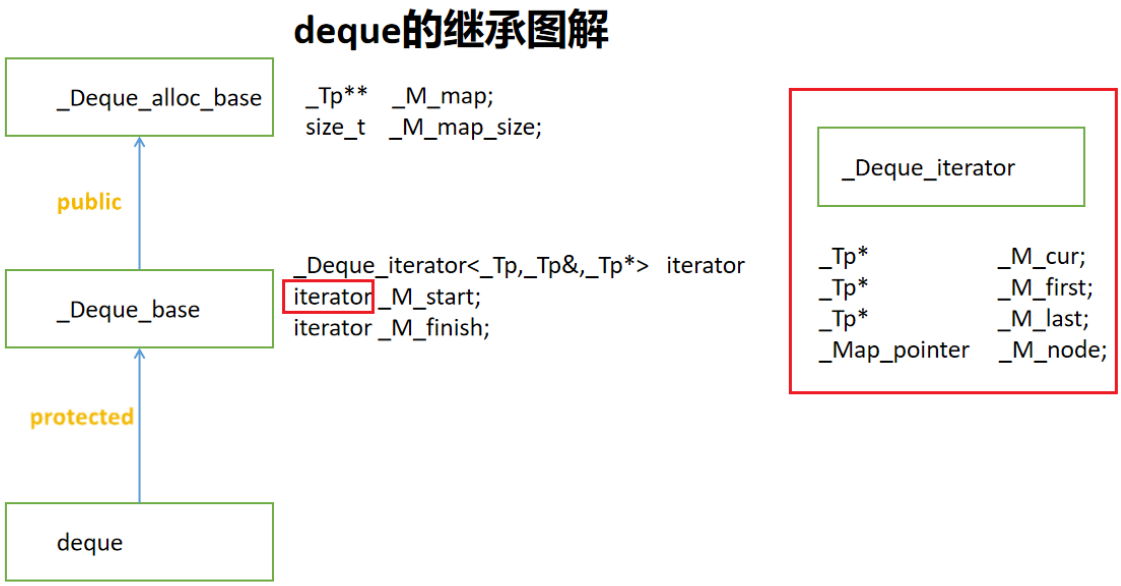
```
for(int idx = 9; idx > 0; idx -= 2)
{
    //9 7 5 3 1 -1
}
```

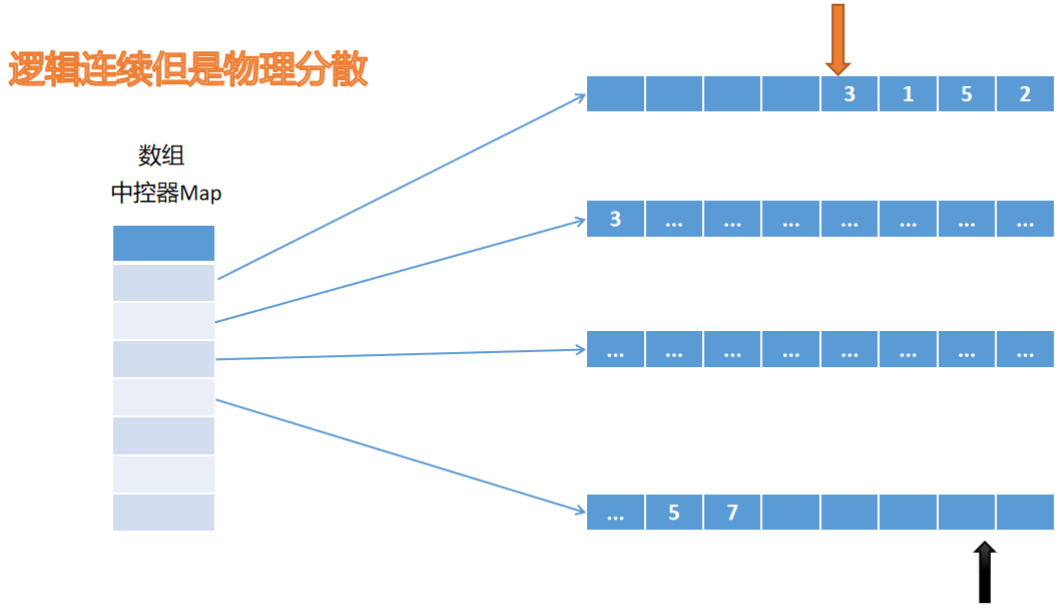
at函数与下标都可以随机访问，但是at有范围检查，比下标更加安全一些。




```
//Q:获取vector中第一个元素的地址
&number;//error
number[0];//ok
&*number.begin();//ok
int *pdata = number.data();//ok
```

3、deque的源码





4、在任意位置插入元素

```
39    cout << endl << "在list任意位置进行插入与删除" << endl;
40    list<int>::iterator it = number.begin();
41    ++it;
42    ++it;
43    cout << "*it = " << *it << endl;
44    number.insert(it, 111);  1、找一个位置插入一个元素
45    display(number);
46    cout << "*it = " << *it << endl;
47
48    cout << endl;
49    number.insert(it, 5, 33);  2、找一个位置插入count个元素
50    display(number);
51    cout << "*it = " << *it << endl;
52
53    cout << endl;
54    vector<int> vec = {12, 34, 56, 78};
55    number.insert(it, vec.begin(), vec.end());
56    display(number);  3、找一个位置插入迭代器范围的元素
57    cout << "*it = " << *it << endl;
```

```
59    cout << endl;        4、插入大括号范围的元素
60    number.insert(it, {111, 222, 333, 444});
61    display(number);
62    cout << "*it = " << *it << endl;
```

5、vector迭代器失效（重要）

在vector使用insert进行插入元素的时候，因为插入元素的**个数是不一定的**，所以有可能导致**底层发生扩容现象**，而如果没有重置迭代器，就有可能发生迭代器指向老的空间的现象，而下一次操作的时候是新的空间。为了解决这个问题，**每次使用迭代器的时候，将迭代器重新置位即可**（也就是让迭代器在使用的时候，与新的空间产生联系即可）

```
    cout << endl;
    it = number.begin();
    ++it;
    ++it;
    /* it = it + 2; */
    cout << "*it = " << *it << endl;
    number.insert(it, {111, 222, 333, 444});
    display(number);
    cout << "*it = " << *it << endl;
    cout << "number.size() = " << number.size() << endl;
    cout << "number.capacity() = " << number.capacity() << endl
#endif
}
```

不论以前底层有没有扩容，都让迭代器重新置位
如果扩容了就与新的空间产生联系；如果没有
扩容还是与老的空间产生联系

6、vector的删除会导致迭代器失效（重要）

```
void test()
{
    vector<int> number = {1, 3, 3, 3, 3, 5, 6, 8, 2};
    /* vector<int> number = {1, 3, 2, 3, 4, 3, 6, 3, 5, 6, 8, 2
    display(number);

    for(auto it = number.begin(); it != number.end(); ++it)
    {
        //不能将所有满足条件的3删除掉(不能删除连续的3)
        if(3 == *it)
        {
            number.erase(it);
        }
    }
    display(number);
}
```

满足条件的时候，删除元素，但是不能
再将迭代器向后移动，因为删除某个元素
之后，后面的元素会自动向前移动

```
void test2()
{
    vector<int> number = {1, 3, 3, 3, 3, 5, 6, 8, 2};
    /* vector<int> number = {1, 3, 2, 3, 4, 3, 6, 3, 5, 6, 8, 2
    display(number);

    for(auto it = number.begin(); it != number.end(); )
    {
        //不能将所有满足条件的3删除掉(不能删除连续的3)
        if(3 == *it)
        {
            number.erase(it);
        }
        else
        {
            ++it;
        }
    }
    display(number);
}
```

此处直接删除元素即可，不用在移动迭代器

不满足条件的时候，再移动迭代器即可

7、元素的清空

对于vector而言，提供了size()函数记录元素的个数，以及capacity()函数记录容量的大小，清空元素的时候，空间是没有回收的，所以可以使用shrink_to_fit()回收多余的空间。对于deque而言，提供了size()函数记录元素的个数的函数，但是没有提供capacity()函数记录容量的大小，但是提供了shrink_to_fit()回收多余的空间；对于list而言，删除元素的时候，也会回收空间，所以没有shrink_to_fit()函数。

```
    cout << endl << "vector清空元素" << endl;
    number.clear();//将元素清空
    number.shrink_to_fit();//回收多余的空间
    cout << "number.size() = " << number.size() << endl;
    cout << "number.capacity() = " << number.capacity() << endl;
```

8、list的特殊使用

sort函数

```
    cout << endl << "测试list的sort函数" << endl;
    /* number.sort(); */
    /* std::less<int> les; */
    /* number.sort(les); */
    /* number.sort(std::less<int>()); */
    /* number.sort(std::greater<int>()); */
    number.sort(CompareList<int>());
    display(number);
```

std::less等价于小于符号

std::greater等价于是大于符号

自定义的比较函数

```
template <typename T>
struct CompareList
{
    bool operator()(const T &lhs, const T &rhs) const
    {
        cout << "struct CompareList" << endl;
        return lhs < rhs;
    }
};

template <>
struct CompareList<int>
{
    bool operator()(const int &lhs, const int &rhs) const
    {
        cout << "struct CompareList<int>" << endl;
        return lhs < rhs;
    }
};
```

重载函数调用运算符

模板的特化

unique函数

```
    cout << endl << "测试list的unique函数"<< endl;
    number.unique();
    display(number);
```

unique去除连续重复的元素，所以可能需要先排序，再去重

merge函数

```
    cout << endl << "测试list的merge函数" << endl;
    /* number.sort(std::greater<int>()); */
    list<int> number2 = {2, 77, 99, 88};
    number2.sort();//将number2先排序
    /* number2.sort(std::greater<int>());//将number2先排序 */
    number.merge(number2);
    display(number);
    display(number2);
```

merge两个链表的时候，需要将两个链表都先按照从小到大进行排序，然后再merge操作

splice函数

```
    cout << endl << "测试list的splice函数" << endl;
    list<int> number3 = {111, 333, 777, 999};
    auto it = number.begin();
    ++it;
    ++it;
    cout << "*it = " << *it << endl;
    auto it3 = number3.end();
    --it3;
    cout << "*it3 = " << *it3 << endl;
    /* number.splice(it, number3); */
    /* display(number); */
    /* display(number2); */
    number.splice(it, number3, it3);
    display(number);
    display(number3);
```

1、把另外一个链表中的所有元素移动到number中

```
    /* display(number2); */
    number.splice(it, number3, it3);
    display(number);
    display(number3);

    cout << endl;
    auto it2 = number3.begin();
    cout << "*it2 = " << *it2 << endl;
    it3 = number3.begin();
    ++it3;
    ++it3;
    cout << "*it3 = " << *it3 << endl;
    number.splice(it, number3, it2, it3);
    display(number);
    display(number3);
```

2、移动另外一个链表中的一个元素

3、移动另外一个链表中的迭代器范围的元素

```
    cout << endl;
    auto it4 = number.begin();
    cout << "*it4 = " << *it4 << endl;
    auto it5 = number.end();
    --it5;
    cout << "*it5 = " << *it5 << endl;
    number.splice(it4, number, it5);
    display(number);
}
```

4、也可以是同一个链表

9、vector的Insert扩容

```

    //因为push_back每次插入元素的个数是固定的，所以按照2倍进行
    //扩容是没有问题的，但是insert操作，每次插入的元素的个数
    //是不一定,所以不能有统一的扩容方法
    //size() = m = 10, capacity() = n = 16, 插入的元素的个数t
    //1、t < n - m时,剩余的容量比要插入的元素大，就不会扩容
    //2、n - m < t < m时,会按照2 * m进行扩容
    //3、n - m < t, m < t < n时,会按照 t + m进行扩容
    //4、n - m < t, n < t时,会按照 t + m进行扩容
    cout << endl;
```

Day22

一、问题回顾

1、序列式容器的初始化方式有哪几种？遍历有哪几种？

2、序列式容器的insert操作需要注意什么？

3、vector的迭代器失效？

4、list的特殊操作？

二、set的使用

```
1、如何将左值转换为右值
int a = 10;
std::move(a);//std::move本质就是做了一个强转，static_cast<T &&>(lvalue)

2、如何将右值转换为左值
int b = std::move(a);
int &&rref = std::move(a);

3、如何缩短变量的生命周期
void test
{
    {
        int a = 10;
    }

    int b = 20;

    int c = 10;
    //...
    //...
    //...
}
```

1、set的特征

```
    //set的特征
    //1、key值是唯一的，不能重复
    //2、默认情况下，会按照key值升序排列
    //3、set的底层实现是红黑树结构
    set<int> number = {1, 2, 5, 9, 8, 6, 4, 2, 2, 6};
    display(number);

```

2、set的查找

```
    cout << endl << "set的查找方法" << endl;
    size_t cnt1 = number.count(2);  查找对应的key值的个数
    size_t cnt2 = number.count(10);
    cout << "cnt1 = " << cnt1 << endl;
    cout << "cnt2 = " << cnt2 << endl;

    cout << endl;
    set<int>::iterator it = number.find(6);  查找key值是否存在set中
    /* auto it = number.find(6); */
    if(it == number.end())
    {
        cout << "该元素是不存在set中" << endl;
    }
    else
    {
        cout <<"该元素存在set中 " << *it << endl;
    }
```

3、insert操作

```
    cout << endl << "set的插入操作" << endl;
    pair<set<int>::iterator, bool> ret = number.insert(7);
    if(ret.second)  1、插入一个元素
    {
        cout << "该元素插入成功 " << *ret.first << endl;
    }
    else
    {
        cout << "该元素已经存在set中，插入失败" << endl;
    }
    display(number);

    cout << endl;
    vector<int> vec = {3, 11, 77, 22};
    number.insert(vec.begin(), vec.end());  2、插入迭代器范围的元素
    display(number);

    number.insert({88, 77, 10, 23});  3、插入大括号的元素
    display(number);
```

4、set不支持下标

```
    cout << endl << "set的下标操作" << endl;
    cout << "number[1] = " << number[1] << endl;
}                                   set是不支持下标访问的

int main(int argc, char **argv)
{
```

```
wangdao@wangdao:CppBase_day22$ g++ 1_set.cc
1_set.cc: In function 'void test()':
1_set.cc:69:37: error: no match for 'operator[]' (operand types are
std::set<int>' and 'int')
     cout << "number[1] = " << number[1] << endl;
                                        ^
wangdao@wangdao:CppBase_day22$
```

5、不支持修改操作

```
    /* cout << endl << "set的下标操作" << endl; */
    /* cout << "number[1] = " << number[1] << endl;//error */
    auto it2 = number.begin();
    cout << "*it2 = " << *it2 << endl;
    /* *it2 = 100;//error,不支持修改 */
}
```

为了维持红黑树的稳定性，不允许set中的元素被修改

6、针对于自定义类型（重点）

```
//命名空间是可以进行扩展的
namespace  std
{
//模板的特化
template <>
struct less<Point>
{
    bool operator()(const Point &lhs, const Point &rhs) const
    {
        //ctrl + v, j,shift + i, tab, esc
        cout << "bool std::less::operator()" << endl;
```

1、第一种形式，可以使用模板的特化形式

```

bool operator<(const Point &lhs, const Point &rhs)
{
    cout << "bool operator<()" << endl;
    if(lhs.getDistance() < rhs.getDistance())
    {
```

2、std::less本质上是一个小于符号，所以就重载小于符号

```

struct ComparePoint
{
    bool operator()(const Point &lhs, const Point &rhs) const
    {
        //ctrl + v, j,shift + i, tab, esc
        cout << "bool ComparePoint" << endl;
         if(lhs.getDistance() < rhs.getDistance())
         {
```

3、自定义比较函数，函数对象对象（仿函数）

三、multiset的使用

1、multiset的特征

```
    //multiset的特征
    //1、key值是不唯一的，可以重复
    //2、默认情况下，会按照key值升序排列
    //3、multiset的底层实现是红黑树结构
    multiset<int> number = {1, 2, 5, 9, 8, 6, 4, 2, 2, 6};
    /* multiset<int, std::greater<int>> number = {1, 2, 5, 9, 8, 6
    display(number);
```

2、其他操作

查找操作count与find与set完全相同，插入操作insert，因为multiset可以存放重复元素，所以就没有返回类型是pair的，另外插入的一个元素、多个元素都与set完全一致。multiset也不支持下标，也不支持修改操作。

3、查找函数

```
    cout << endl << "set的bound函数" << endl;
    auto it3 = number.lower_bound(2);
    cout << "*it3 = " << *it3 << endl;

    auto it4 = number.upper_bound(2);
    cout << "*it4 = " << *it4 << endl;
    while(it3 != it4)
    {
        cout << *it3 << "  ";
        ++it3;
    }

    cout << endl;
    pair<multiset<int>::iterator, multiset<int>::iterator>
        ret = number.equal_range(2);
    while(ret.first != ret.second)
    {
        cout << *ret.first << "  ";
        ++ret.first;
    }
    cout << endl;
```

返回第一个大于等于key值迭代器

返回第一个大于key的迭代器

返回的是一个范围，等价于执行了lower_bound与upper_bound

4、针对于自定义类型

使用方法类似与set容器，可以特化std::less、或者重载小于符号、或者写函数对象的形式。

四、map的使用

1、map的特征

```
    //map的特征
    //1、存放的是键值对形式key-value，key值是唯一的
    //不能重复，但是value值是可以重复的
    //2、默认情况下，会按照key值升序排列
    //3、底层使用的是红黑树
    map<int, string> number = {
        pair<int, string>(2, "beijing"),
        pair<int, string>(7, "nanjing"),
        {3, "wuhan"},
        {3, "wuhan"},
        make_pair(1, "wangdao"),
        make_pair(4, "wangdao"),
    };
    display(number);
```

2、map的操作

```
    cout << endl << "map的查找操作" << endl;
    size_t cnt1 = number.count(3);
    cout << "cnt1 = " << cnt1 << endl;

    cout << endl;
    map<int, string>::iterator it = number.find(10);
    if(it != number.end())
    {
        cout << "该元素存在map中 "
             << it->first << "  "
             << it->second << endl;
    }
    else
    {
        cout << "该元素不存在map中" << endl;
    }
```

3、map的插入操作

```
    cout << endl << "map的insert操作" << endl;
    pair<map<int, string>::iterator, bool>
        /* auto */
        /* ret = number.insert(pair<int, string>(5, "hubei")); */
        /* ret = number.insert({5, "hubei"}); */
        ret = number.insert(make_pair(5, "hubei"));
    if(ret.second)
    {
        cout << "插入成功 "
             << ret.first->first << "  "
             << ret.first->second << endl;
    }
    else
    {
        cout << "插入失败，该元素存在map中" << endl;
    }
    display(number);

    cout << endl << endl;
    number.insert({{1, "wangdao"}, {8, "dongjing"}, {10, "huangshi"}});
    display(number);
```

4、下标（重要）

```
templete <typename Container>
void fun(const Container &con)
{
    cout << con[2] << endl;
}
```

```
    cout << endl << "map的下标" << endl;
    //下标中传递的是key值，如果key存在，会返回value值，如果key
    //不存在，会返回空
    cout << "number[1] = " << number[1] << endl;//查找
    cout << "number[6] = " << number[6] << endl;//插入
    display(number);

    cout << endl << endl;
    //T &operator[](const Key &key)
    number[6] = "wuhan";//修改
    number[5] = "wuhan";
    display(number);
```

```

    /* map<int, string> mm = {1, "wangdao"); */
    const map<int, string> mm = {
        pair<int, string>(2, "beijing"),
    };
    mm[2];
}
```

map支持下标访问，但是下标访问是没有const版本的，所以一个const的map对象是不能调用下标的

5、针对于自定义类型

```
template<
    class Key,
    class T,
    class Compare = std::less<Key>,
    class Allocator = std::allocator<std::pair<const Key, T> >
> class map;
```

```
namespace pmr {
```

如果map的key是一个自定义类型的时候，需要针对自定义类型实现std::less，否则会报错。

注意：std::less针对的是key类型，而不是pair类型

五、multimap的使用

1、multimap的特征

```
    //multimap的特征
    //1、存放的是键值对形式key-value，key值是不唯一的
    //可以重复，value值可以重复，也可以不重复
    //2、默认情况下，会按照key值升序排列
    //3、底层使用的也是红黑树
    multimap<int, string> number = {
        pair<int, string>(2, "beijing"),
        pair<int, string>(7, "nanjing"),
        {3, "wuhan"},
        {3, "wuhan"},
        make_pair(1, "wangdao"),
        make_pair(4, "wangdao"),
    };
    display(number);
```

2、其他操作

multimap的查找操作count与find与map是完全一样，插入操作insert基本与map一致。

3、不支持下标访问

```
    cout << "number[1] = " << number[1] << endl;//查找
    cout << "number[6] = " << number[6] << endl;//插入
    display(number);
```

4、针对于自定义类型

与map是一样的。

六、哈希相关的概念

1、哈希函数

通过key值找到对应的位置值。

```
index = H(key)
```

2、构建方法

除留取余法：H(key) = key mod p (p <= m, m为表长)

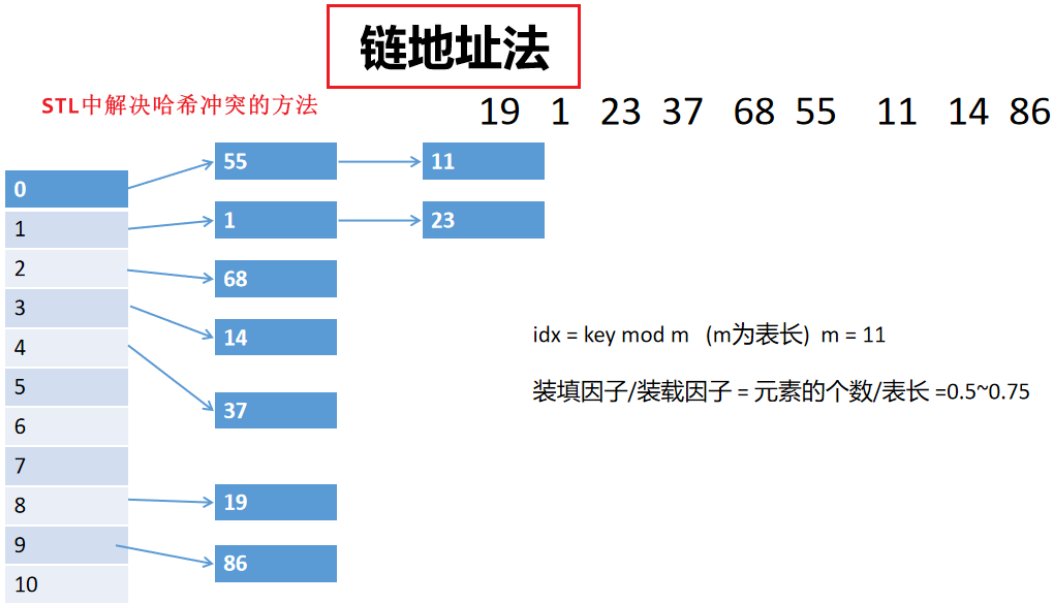
3、哈希冲突（重要）

不同的key值，经过哈希函数之后，对应的位置值是一样的，称为哈希冲突

```
H(key1) = H(key2)   (key1 != key2)
```

4、解决哈希冲突

链地址法



5、装载因子

装载因子 a = (实际装载数据的长度n)/(表长m)

七、unordered_set的使用

1、基本特征

```
//unordered_set的特征
//1、key值是唯一的，不能重复
//2、key值是没有顺序的
//3、底层使用的是哈希
unordered_set<int> number = {1, 3, 7, 9, 4, 3, 8, 3, 7};
display(number);
```

2、针对于自定义类型(重要)

```
template<
    class Key,
    class Hash = std::hash<Key>,
    class KeyEqual = std::equal_to<Key>,
    class Allocator = std::allocator<Key>
> class unordered_set;
```

模板参数Hash的设计

```
//命令空间是可以进行扩展的
namespace  std
{
//模板的特化
template <>
class hash<Point>  1、hash针对Point实现模板的特化版本
{
public:
    size_t operator()(const Point &rhs) const
    {
        cout << "std::hash" << endl;
        return (rhs.getX() << 1) ^ (rhs.getY() << 2);  实现哈希函数
    }
};

}//end of namespace std
```

```
//自定义哈希函数，函数对象的形式  2、函数对象的形式
struct HashPoint
{
    size_t operator()(const Point &rhs) const
    {
        cout << "size_t HashPoint::operator()" << endl;
        return (rhs.getX() << 1) ^ (rhs.getY() << 2);
    }

};
```

模板参数equal_to的设计

因为equal_to代表的是等号，也就是一个运算符，所以会多一个运算符的重载。

```
//命令空间是可以进行扩展的
namespace  std
{
//模板的特化
template <>
class equal_to<Point>  1、模板的特化
{
public:
    bool operator()(const Point &lhs, const Point &rhs) const
    {
        cout << "std::equal_to" << endl;
        return (lhs.getX() == rhs.getX()) && (lhs.getY() == rhs.getY());
    }
};

}//end of namespace std
```

```
struct EqualToPoint  2、函数对象的形式
{
    bool operator()(const Point &lhs, const Point &rhs) const
    {
        cout << "struct EqualToPoint" << endl;
        return (lhs.getX() == rhs.getX()) && (lhs.getY() == rhs.getY());
    }

};
```

```
3、因为equal_to等价于等号，所以可以使用运算符重载的形式，比较两个Point
bool operator==(const Point &lhs, const Point &rhs)
{
    cout << "bool operator==" << endl;
    return (lhs.getX() == rhs.getX()) && (lhs.getY() == rhs.getY());
}
```

八、unordered_multiset的使用

1、基本特征

```
void test()
{
    //unordered_multiset的特征
    //1、key值是不唯一的，可以重复
    //2、key值是没有顺序的
    //3、底层使用的是哈希
    unordered_multiset<int> number = {1, 3, 7, 9, 4, 3, 8, 3, 7};
    display(number);
}
```

2、针对于自定义类型

基本使用方法与unordered_set完全一致。

Day23

一、问题回顾

1、关联式容器的特点?

元素是排好序的、红黑树。

2、关联式容器的基本操作：初始化、遍历、查找（两个）、插入、删除、清空、交换操作？哪些具备下标?

map具备下标

3、针对自定义类型的方式有哪些？（重要）

模板的特化、函数对象形式（自定义函数对象）、进行运算符重载

4、无序关联式容器的底层实现是什么?

哈希

二、unordered_map的使用

1、基本特征

```
    //unordered_map的特征
    //1、存放是key-value类型，key值是唯一的，不能重复
    //但是value值既可以重复也可以不重复
    //2、key值是没有顺序的
    //3、底层使用的还是哈希
    unordered_map<int, string> number = {
        pair<int, string>(1, "beijing"),
        pair<int, string>(2, "beijing"),
        {5, "nanjing"},
        {7, "wuhan"},
        make_pair(3, "wangdao"),
        make_pair(3, "wangdao"),
    };
    display(number);
```

2、下标

```
    cout << endl << "下标操作" << endl;
    cout << "number[3] = " << number[3] << endl;//查找
    cout << "number[6] = " << number[6] << endl;//插入
    display(number);

    cout << endl <<endl;
    number[6] = "point";//修改
    display(number);
```

3、针对于自定义类型

常规的数据类型都已经做了std::hash的特化，string也做了hash的特化。

```
template<> struct hash<bool>;
template<> struct hash<char>;
template<> struct hash<signed char>;
template<> struct hash<unsigned char>;
template<> struct hash<char16_t>;
template<> struct hash<char32_t>;
template<> struct hash<wchar_t>;
template<> struct hash<short>;
template<> struct hash<unsigned short>;
template<> struct hash<int>;
template<> struct hash<unsigned int>;
template<> struct hash<long>;
template<> struct hash<long long>;
template<> struct hash<unsigned long>;
template<> struct hash<unsigned long long>;
template<> struct hash<float>;
template<> struct hash<double>;
template<> struct hash<long double>;
template< class T > struct hash<T*>;
```

```
std::hash<std::string>          (C++11)
std::hash<std::u8string>        (C++20)
std::hash<std::u16string>       (C++11)
std::hash<std::u32string>       (C++11)
std::hash<std::wstring>         (C++11)  hash support for strings
std::hash<std::pmr::string>     (C++20)  (class template specialization)
std::hash<std::pmr::u8string>   (C++20)
std::hash<std::pmr::u16string>  (C++20)
std::hash<std::pmr::u32string>  (C++20)
std::hash<std::pmr::wstring>    (C++20)
```

三、unordered_multimap的使用

1、基本特征

```
    //unordered multimap的特征
    //1、存放是key-value类型，key值是不唯一的，可以重复
    //但是value值既可以重复也可以不重复
    //2、key值是没有顺序的
    //3、底层使用的还是哈希
    unordered_multimap<int, string> number = {
        pair<int, string>(1, "beijing"),
        pair<int, string>(2, "beijing"),
        {5, "nanjing"},
        {7, "wuhan"},
        make_pair(3, "wangdao"),
        make_pair(3, "wangdao"),
    };
    display(number);
```

2、不支持下标访问的

```
#if 0
    cout << endl << "下标操作" << endl;
    cout << "number[3] = " << number[3] << endl;//查找
    cout << "number[6] = " << number[6] << endl;//插入
    display(number);
```

不支持下标访问

3、针对于自定义类型

与unordered_map完全一致。

四、优先级队列

```
Defined in header <queue>
template<
    class T,
    class Container = std::vector<T>,
    class Compare = std::less<typename Container::value_type>
> class priority_queue;
```

```
{
    //优先级队列的底层使用的是堆排序（默认是大根堆）
    //当有元素插入进来的时候，会与堆顶进行比较，如果堆顶比
    //新插入的元素小的话，那么就满足std::less,就会将新插入
    //的元素作为新的堆顶；如果新插入的元素与堆顶进行比较，
    //堆顶比新插入的元素大，也就不满足std::less，那么老的
    //堆顶依旧还是在堆顶的位置，不会发生置换    原理
    vector<int> vec = {1, 4, 7, 9, 5, 6, 8};
    /* priority_queue<int> pque(vec.begin(), vec.end()); */
    priority_queue<int> pque;  1、初始化

    for(size_t idx = 0; idx != vec.size(); ++idx)
    {
        pque.push(vec[idx]);
        cout << "优先级最高的元素是 : " << pque.top() << endl;
    }

    while(!pque.empty())
    {     2、遍历的方法
        cout << pque.top() << "  ";
        pque.pop();
    }
```

五、容器的选择(重点)

1、元素是有序的

关联式容器中的元素默认是有序的，无序关联式容器中的元素是没有顺序的。

2、容器是否有下标

序列式容器：vector、deque。

关联式容器：map

无序关联式容器：unoredered_map

3、时间复杂度

序列式容器：O(N)

关联式容器：O(logN)

无序关联式容器：O(1)

4、迭代器的类型

随机访问迭代器：vector、deque
双向迭代器：list、关联式容器
前向迭代器：无序关联式容器

5、是否所有的容器都具备迭代器

容器适配器是没有迭代器

Day24

一、迭代器

1、概念

概念：迭代器可以看成是一种指针。具备普通指针的功能。

本质：是种更高层次的抽象，它使得算法独立于容器

2、类型

迭代器类型

- 不同的算法对迭代器的要求不同，为此，STL定义了5种迭代器，分别是
  - 随机访问迭代器（RandomAccessIterator）
  - 双向迭代器（BidirectionalIterator）
  - 前向迭代器（ForwardIterator）
  - 输出迭代器（OutputIterator）
  - 输入迭代器（InputIterator）
- 其层次结构如所示。

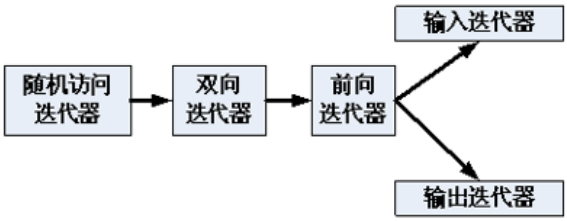


图 13.1　迭代器层次结构图

3、输出流迭代器

std::ostream_iterator<T,CharT,Traits>::ostream_iterator

`ostream_iterator(ostream_type& stream, const CharT* delim)`	(1)
`ostream_iterator(ostream_type& stream)`	(2)

```
class ostream_iterator
{
public:
    //ostream_iterator<int> osi(cout, "\n");
    //ostream_type& __s = cout
    //const _CharT* __c = "\n";
    //_M_stream = &cout;
    //_M_string = "\n"
    ostream_iterator(ostream_type& __s, const _CharT* __c)
    : _M_stream(&__s)
    , _M_string(__c)
    {}
```

```

    ostream_iterator<_Tp>& operator=(const _Tp& __value) {
    *_M_stream << __value;
    if (_M_string) *_M_stream << _M_string;
    return *this;
  }

    ostream_iterator<_Tp>& operator*()
    {
        return *this;
    }
    ostream_iterator<_Tp>& operator++()
    {
        return *this;
    }
    ostream_iterator<_Tp>& operator++(int)
    {
        return *this;
    }
private:
  ostream_type* _M_stream;
  const _CharT* _M_string;

};

_Trivial:平凡的、琐碎的。
POD类型。

template <class _InputIter, class _OutputIter>
inline _OutputIter copy(_InputIter __first, _InputIter __last,
                        _OutputIter __result) {
  __STL_REQUIRES(_InputIter, _InputIterator);
  __STL_REQUIRES(_OutputIter, _OutputIterator);
  return __copy_aux(__first, __last, __result, __VALUE_TYPE(__first));
}

template <class _InputIter, class _OutputIter, class _Tp>
inline _OutputIter __copy_aux(_InputIter __first, _InputIter __last,
                              _OutputIter __result, _Tp*) {
  typedef typename __type_traits<_Tp>::has_trivial_assignment_operator
          _Trivial;
  return __copy_aux2(__first, __last, __result, _Trivial());
}

template <class _InputIter, class _OutputIter>
inline _OutputIter __copy_aux2(_InputIter __first, _InputIter __last,
                               _OutputIter __result, __true_type) {
  return __copy(__first, __last, __result,
                __ITERATOR_CATEGORY(__first),
                __DISTANCE_TYPE(__first));
}

template <class _InputIter, class _OutputIter, class _Distance>
inline _OutputIter __copy(_InputIter __first, _InputIter __last,
                          _OutputIter __result,
                          input_iterator_tag, _Distance*)
{
  for ( ; __first != __last; ++__result, ++__first)
```

```
    *__result = *__first;
  return __result;
}

// __first = vec.begin()
//__last = vec.end()
// __result = osi
//vector<int> vec = {1, 4, 7, 9, 3};
copy(vec.begin(), vec.end(), osi);
              last
1, 4, 7, 9, 3
               f

*osi = 7 ;


ostream_iterator<_Tp>& operator=(const _Tp& __value)
{
    cout << 7;
    if (_M_string) cout << "\n";
    return *this;
  }
```

4、输入流迭代器

std::istream_iterator<T,CharT,Traits,Distance>::istream_iterator

`constexpr istream_iterator();`	(1)
`istream_iterator( istream_type& stream );`	(2)
`istream_iterator( const istream_iterator& other ) = default;`	(3)

```
class istream_iterator
{
public:
    //istream_iterator<int> isi(std::cin);
    //istream_type& __s = cin;
    //_M_stream  = &cin;
    istream_iterator(istream_type& __s)
    : _M_stream(&__s)
    {
        _M_read();
    }

    istream_iterator()
    : _M_stream(0)
     , _M_ok(false) {}

    istream_iterator& operator++()
    {
        _M_read();
        return *this;
    }

    reference operator*() const
    {
        return _M_value;
```

```
    }

    bool _M_equal(const istream_iterator& __x) const
    {
        return (_M_ok == __x._M_ok) && (!_M_ok || _M_stream == __x._M_stream);
    }
private:
   istream_type* _M_stream;
   _Tp _M_value;
   bool _M_ok;

    if(cin)//operator bool(){}
    {

    }

     void _M_read()
     {
         _M_ok = (_M_stream && *_M_stream) ? true : false;//三目运算符
         if (_M_ok)
         {
             *_M_stream >> _M_value;//cin >> value
             _M_ok = *_M_stream ? true : false;
         }
     }

};


//__x1 = isi
//__y = istream_iterator<int>()
inline bool operator!=(const istream_iterator<_Tp, _CharT, _Traits, _Dist>& __x1,
                       const istream_iterator<_Tp, _CharT, _Traits, _Dist>& __y)
{
  return !__x1._M_equal(__y);
}

bool _M_equal(const istream_iterator& __x) const
{
    return (_M_ok == __x._M_ok) && (!_M_ok || _M_stream == __x._M_stream);
}

template <class _Container>
inline back_insert_iterator<_Container> back_inserter(_Container& __x) {
  return back_insert_iterator<_Container>(__x);
}


copy(isi, istream_iterator<int>(), std::back_inserter(vec));

//_InputIter __first = isi;
// _InputIter __last = istream_iterator<int>()
//_OutputIter __result = std::back_inserter(vec)
inline _OutputIter __copy(_InputIter __first, _InputIter __last,
                          _OutputIter __result,
```

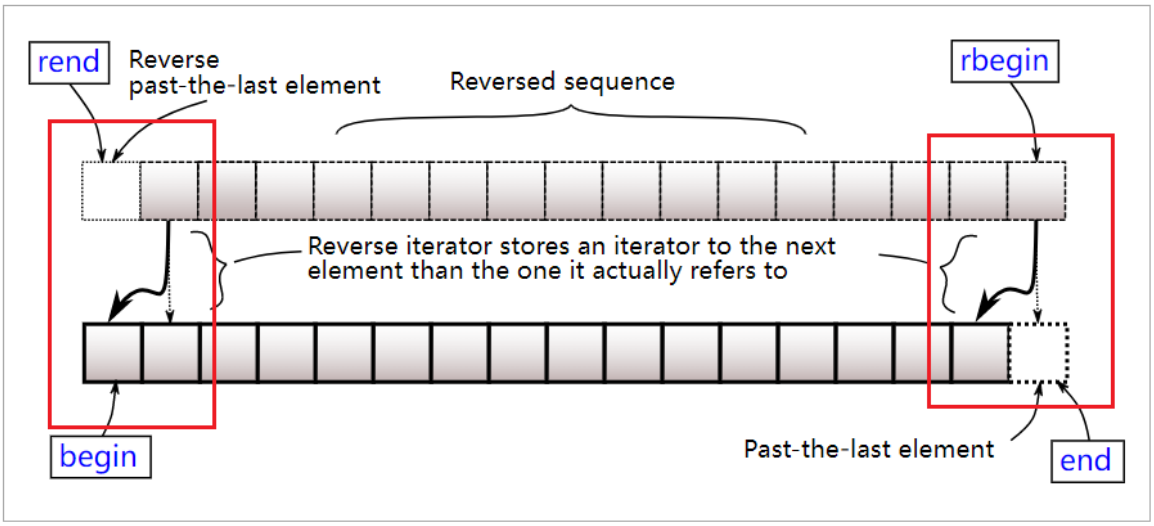
```
                                input_iterator_tag, _Distance*)
{
  for ( ; __first != __last; ++__result, ++__first)
    *__result = *__first;
  return __result;
}

*__result = 2;



class back_insert_iterator {
public:
    back_insert_iterator<_Container>&
    operator=(const typename _Container::value_type& __value)
    {
        container->push_back(__value);//调用了vector放入push_back将数据value插入到尾部
        return *this;
    }

    back_insert_iterator<_Container>& operator*()
    {
        return *this;
    }

    back_insert_iterator<_Container>& operator++() { return *this; }
    back_insert_iterator<_Container>& operator++(int) { return *this; }
}
```

5、迭代器适配器

back_insert_iterator	iterator adaptor for insertion at the end of a container (class template)
back_inserter	creates a std::back_insert_iterator of type inferred from the argument (function template)
front_insert_iterator	iterator adaptor for insertion at the front of a container (class template)
front_inserter	creates a std::front_insert_iterator of type inferred from the argument (function template)
insert_iterator	iterator adaptor for insertion into a container (class template)
inserter	creates a std::insert_iterator of type inferred from the argument (function template)

```
    //将vector中的元素插入到list头部
    //std::front_inserter以及front_insert_iterator底层会
    //调用push_front
    /* copy(vecNumber.begin(), vecNumber.end(), */
    /*      front_inserter(listNumber)); */
    copy(vecNumber.begin(), vecNumber.end(),
         front_insert_iterator<list<int>>(listNumber));
    copy(listNumber.begin(), listNumber.end(),
         ostream_iterator<int>(cout, "  "));
    cout << endl;
```

6、反向迭代器




```
void test2()
{
    vector<int> number = {1, 3, 5, 7, 9, 2, 4, 6, 8};
    vector<int>::reverse_iterator rit;
    for(rit = number.rbegin(); rit != number.rend(); ++rit)
    {
        cout << *rit << "  ";
    }
    cout << endl;
}
```

二、算法

算法库中的函数都是非成员函数（自由函数、全局函数）

1、分类

1、非修改式的算法：**for_each**、count、find

2、修改式的算法：**copy**、**remove_if**、swap

3、排序算法：**sort**

4、二分搜索：lower_bound、upper_bound

5、集合操作：set_intersection

6、堆相关的操作：make_heap

7、取最值：max、min

8、未初始化的操作：uninitialized_copy

2、for_each算法

一元函数：函数的参数只有一个。二元函数：函数的参数是两个。

一元谓词/断言：函数的参数只有一个，并且函数的返回类型是bool。

二元谓词/断言：函数的参数有两个，并且函数的返回类型是bool。

```
void test()
{
    int a = 10;
    vector<int> number = {1, 3, 9, 7, 6};
    /* for_each(number.begin(), number.end(), func); */
    //lambda表达式,匿名函数
    //[], 捕获列表
    /* for_each(number.begin(), number.end(), [&a](int &value){ */
    /* for_each(number.begin(), number.end(), [](int &value) mutable
    for_each(number.begin(), number.end(), [](int &value) {
            /* cout << a << endl; */
            /* ++a; */
            ++value;
            cout << value << "  ";
            });
    cout << endl;
    copy(number.begin(), number.end(),
         ostream_iterator<int>(cout, "  "));
    cout << endl;
    cout << a << endl;
}
```

3、remove_if算法

```
    //将所有大于6的元素删除
    //remove_if可以配合erase进行使用，达到删除满足条件的元素
    /* auto it = remove_if(number.begin(), number.end(), func); */
    number.erase(it, number.end());
    copy(number.begin(), number.end(),
         ostream_iterator<int>(cout, "  "));
```

```
template<class ForwardIt, class UnaryPredicate>
ForwardIt remove_if(ForwardIt first, ForwardIt last, UnaryPredicate p)
{
    first = std::find_if(first, last, p);
    if (first != last)
        for(ForwardIt i = first; ++i != last; )
            if (!p(*i))
                *first++ = std::move(*i);
    return first;
}

template<class InputIt, class UnaryPredicate>
constexpr InputIt find_if(InputIt first, InputIt last, UnaryPredicate p)
{
    for (; first != last; ++first) {
        if (p(*first)) {
            return first;
        }
    }
    return last;
}


bool func(int value)
{
```

```
     return value > 6;
}

remove_if(number.begin(), number.end(), func);
vector<int> number = {2, 4, 6, 9, 8, 5, 3};
//ForwardIt first = number.begin();
//ForwardIt last = number.end()
//p = func
ForwardIt remove_if(ForwardIt first, ForwardIt last, UnaryPredicate p)
{
    first = std::find_if(first, last, p);
    if (first != last)
    {
         for(ForwardIt i = first; ++i != last; )
         {
               if (!p(*i))
               {
                   *first++ = std::move(*i);
               }
         }
    }

    return first;
}
         f
2, 4, 6, 9, 8, 5, 3
               i
               f      last
2, 4, 6, 5, 3, 5, 3
                      i

constexpr InputIt find_if(InputIt first, InputIt last, UnaryPredicate p)
{
    for (; first != last; ++first) {
        if (p(*first)) {
            return first;
        }
    }
    return last;
}

//有一种方式，可以将函数的某一个参数固定，那么就可以将二元函数转变为一元函数
bool func1(int x,  int y = 6)====>bool func1(int x) { return x > 6;}
{
    return x > y;
}


bool func2(int x = 6,  int y )====>bool func2(int y) { return 6 < y;}
{
    return 6 < y;
}
```

bind1st(func, 6)====>func(int x, int y)===>func(6, int y)
bind2nd(func2, 6)===>func2(int x, int y)===func2(int x, 6)

```
    /* auto it = remove_if(number.begin(), number.end(), */
    /*                     bind1st(std::greater<int>(), 6));//error */

    auto it = remove_if(number.begin(), number.end(),
                        bind2nd(std::greater<int>(), 6));
                        3、将二元断言固定一个参数，改为一元断言
    /* auto it = remove_if(number.begin(), number.end(), */
    /*                     bind1st(std::less<int>(), 6)); */

    /* auto it = remove_if(number.begin(), number.end(), [](int value){
    /*                     return value > 6; */        2、lambda表达式
    /*                     }); */

    //将所有大于6的元素删除                                1、直接传一元断言
    //remove_if可以配合erase进行使用，达到删除满足条件的元素
    /* auto it = remove_if(number.begin(), number.end(), func); */
```

4、迭代器失效

```
void test()
{
    vector<int> number;
    number.push_back(1);

    bool flag = true;
    for(auto it = number.begin(); it != number.end(); ++it)
    {
        cout << *it << "  ";
        if(flag)
        {
            //底层已经发生了扩容，导致迭代器it失效了
            number.push_back(2);
            it = number.begin();//将迭代器重新置位即可
            flag = false;
        }
    }
    cout << endl;
}
```

5、bind12的使用

std::bind1st, std::bind2nd

bind1st或者bind2nd可以绑定二元函数对象，并将x固定为二元函数对象的第一个参数或者第二个参数，从而变为一元断言

Defined in header <functional>

`template< class F, class T >` `std::binder1st<F> bind1st( const F& f, const T& x );`	(1)	(deprecated in C++11) (removed in C++17)
`template< class F, class T >` `std::binder2nd<F> bind2nd( const F& f, const T& x );`	(2)	(deprecated in C++11) (removed in C++17)

Binds a given argument x to a first or second parameter of the given binary function object f. That is, stores x within the resulting wrapper, which, if called, passes x as the first or the second parameter of f.

三、bind的使用

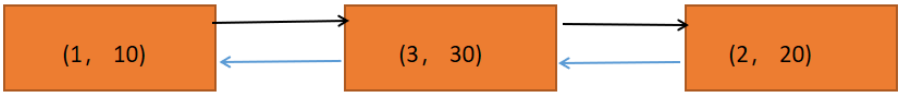
```
    Test tst;                                   可以传对象的地址，取对象地址
    auto f2 = bind(&Test::add, &tst, 1, 2);
    cout << "f2() = " << f2() << endl;
                                                可以传对象本身，拷贝操作
    auto f3 = bind(&Test::add, tst, 10, 20);
    cout << "f3() = " << f3() << endl;
```

Day25

一、作业

```
ifstream ifs(filename);
if(!ifs)
{
    //....异常处理
}

//1、读一行，然后将一行的内容交给istringstream进行处理
string line;
while(getline(ifs, line))
{
    istringstream iss(line);//istringstream属于串IO
    string word;
    while(iss >> word)
    {
        //对word进行处理
    }
}

//2、直接使用ifs进行读操作
string word;
while(ifs >> word)//磁盘IO的次数就比较多
{
    //对word进行处理
}

//3、一次将文件中的内容读到内存中
//打开文件的时候，可以使用追加模式
size_t len = ifs.tellg();
char buf[10000] = {0};//string buf;  buf.c_str();
ifs.read(buf, len);
istringstream iss(buf);//istringstream属于串IO
string word;
while(iss >> word)
{
    //对word进行处理
}


ifs.close();
```

LRU实现原理图




```
struct CacheNode
{
    int key;
    int value;
};
```

list<CacheNode> _nodes; //节点放在list链表之中
unordered_map<int, list<CacheNode>::iterator> _cache;//存储key值以及在链表中的位置

二、问题回顾

1、迭代器的种类？输出流迭代器与输入流迭代器的源码阅读？三对插入迭代器？

2、for_each算法与remove_if算法的传参，以及使用方法？

3、lambda表达式的使用？（C++11中的语法特性）

三、函数指针（重要）

```
void test2()
{
    /* func1(); */
    //延迟调用
    //函数指针
    typedef int (*pFunc)();
    pFunc f1 = &func1;//注册回调函数
    //.....
    //...
    cout << "f1() = " << f1() << endl;//执行回调函数

    f1 = func2;
    cout << "f1() = " << f1() << endl;

    /* f1 = func3;//int(int),error */

}
```

函数指针一旦定义好了之后，就只能指向某种固定类型的函数，不能指向其他类型的函数

1、函数指针可以与回调函数一起使用，

回调函数的使用：包括回调函数的注册与回调函数的执行

2、延迟调用的思想

四、bind的使用（重点）

bind的占位符概念

```
void test2()
{
    int number = 30;
    using namespace std::placeholders;
    //占位符本身代表的是形参的位置
    //占位符中的数字代表的是实参的位置
    //bind默认采用的是值传递
    //引用的包装器,std::cref
    //引用的包装器,std::ref
    auto f = bind(&func, 10, _3, _1,std::cref(number), number);
    number = 200;
    f(20, 50, 70, 90, 300, 500, 1000);//其他没有使用的参数直接丢掉
}
```

bind绑定到数据成员

```

    function<int()> f3 = bind(&Test::data, &tst);
    cout << "f3() = " << f3() << endl;

```

bind除了可以绑定到函数也可以绑定到数据成员

五、function的使用（重要）

std::function

Defined in header <functional>

```
template< class >
class function; /* undefined */
```
(since C++11)

```
template< class R, class... Args >
class function<R(Args...)>;
```
(since C++11)

```
    //函数的形态（函数的类型）：函数的返回类型 + 函数的参数列表
    //函数的参数列表：函数的参数的个数、参数的类型、参数的顺序
    //bind可以改变函数的形态(只能改变函数的参数个数)
    //f的类型：int()
    //function可以存函数的类型，所以将function称为函数的容器
    function<int()> f = bind(&add, 1, 2, 3);
    cout << "f() = " << f() << endl;

    Test tst;
    //f2的类型:int()
    function<int()> f2 = bind(&Test::add, &tst, 1, 2);
    cout << "f2() = " << f2() << endl;
```

六、左值与右值

```
int number = 10;
int &ref = number;//ok,左值引用，可以绑定到左值，不能绑定到右值
int &ref1 = 10;//error
```

```
const int &ref2 = 10;//ok
const int &ref3 = number;//ok
//const左值引用既可以绑定到左值，也可以绑定到右值。

//C++11里面提出了右值引用
int &&rref = 10;//ok,右值引用可以绑定到右值
int &&rref1 = number;//error
//结论：右值引用可以绑定到右值，但是不能绑定到左值。


int &&func()
{
    return 10;
}

class String
{
public:
    String(const String &rhs)
    : _pstr(new char[strlen(rhs._pstr) + 1]())
    {

    }


    String(String &&rhs)
    : _pstr(rhs._pstr)
    {
        rhs._pstr = nullptr;
    }

    //赋值运算符函数
    String &operator=(const String &rhs)
    {
        if(this != &rhs)
        {
            delete [] _pstr;
            _pstr = nullptr;

            //深拷贝
            _pstr = new char[strlen(rhs._pstr) + 1]();
            strcpy(_pstr, rhs._pstr);
        }

        return *this;
    }

    //移动赋值函数
    String &operator=(String &&rhs)//String &&rhs = std::move(cb)
    {
        if(this != &rhs)
        {
            delete [] _pstr;
            _pstr = nullptr;

            //浅拷贝
            _pstr = rhs._pstr;
            rhs._pstr = nullptr;
```

```
        }

        return *this;
    }

private:
    char *_pstr;
}

String s1 = String("hello");
```

七、bind与function结合使用

```
{
public:
    using DisplayCallback = function<void()>;
    /* typedef function<void> DisplayCallback; */
    using AreaCallback = function<double()>;

    DisplayCallback _displayCallback;
    AreaCallback _areaCallback;

    //回调函数的注册与执行
    //注册
    void setDisplayCallback(DisplayCallback &&cb)//移动彻底
    {
        /* _displayCallback = cb; */
        _displayCallback = std::move(cb);
    }

    void setAreaCallback(AreaCallback &&cb)
    {
        _areaCallback = std::move(cb);
    }
```

```
    //执行回调函数
    void handleDisplayCallback() const
    {
        if(_displayCallback)
        {
            _displayCallback();//执行回调函数
        }
    }

    double handleAreaCallback() const
    {
        if(_areaCallback)
        {
            return _areaCallback();
        }
        else
        {
            return 0;
        }
    }
```

八、成员函数适配器mem_fn

```
/* for_each(vec.begin(), vec.end(), mem_fn(&Number::print)); */
Number number;
using namespace std::placeholders;
/* std::function<void()> f = std::bind(&Number::print, &number); */
/* for_each(vec.begin(), vec.end(), f);//error */
for_each(vec.begin(), vec.end(),
/*          std::bind(&Number::print, &number)); */
         std::bind(&Number::print, _1));
cout << endl;
```

九、函数对象（仿函数）

所有与小括号结合，体现函数的含义的都称为函数对象

- 普通函数名
- 函数指针
- 重载了函数调用运算符的类创建的对象

十、空间配置器（重要）

std::allocator

Defined in header <memory>

`template< class T >` `struct allocator;`	(1)	
`template<>` `struct allocator<void>;`	(2)	(deprecated in C++17) (removed in C++20)

会将空间的申请与释放以及对象的构建与销毁严格的分开。

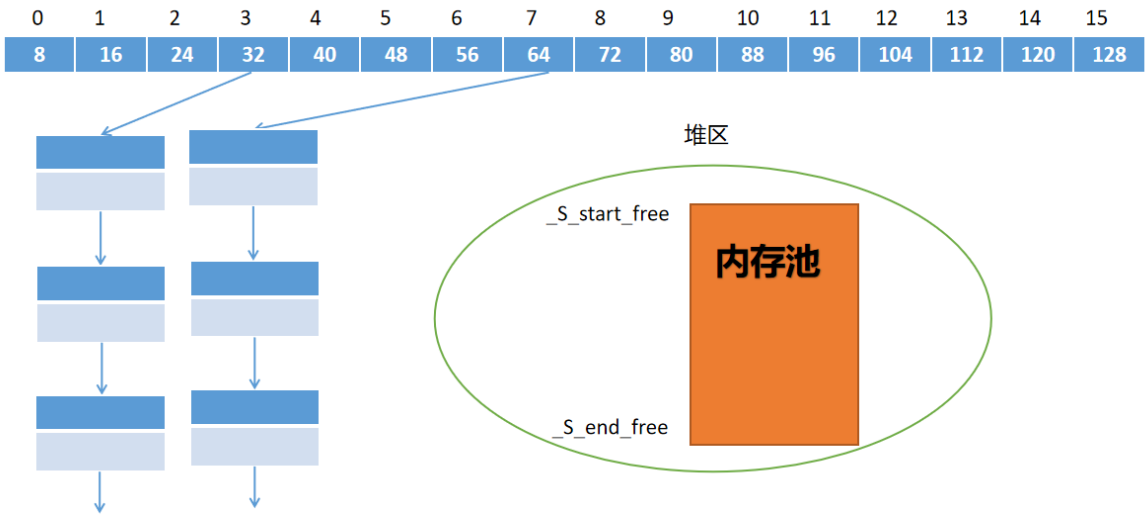
在STL中，对象往往不是一个一个申请，而是大批量申请的，所以不能创建一个对象就申请一片空间，这样空间的申请就比较频繁。

```
Point pt(1, 2);
```

四个重要函数

```
#include <memory>

//空间的申请
T* allocate( std::size_t n );

//空间的释放
void deallocate( T* p, std::size_t n );

//对象的构建
void construct( pointer p, const_reference val );

//对象的销毁
void destroy( pointer p );
```

第二级空间配置器的原理图




```
class Base
{
    virtual print()
    {

    }

    void func1()
    {
        this->print();
    }

    void func2()
    {
        Base::print();
    }
}


class Derived: public Base
```

```
{
    virtual print()
    {

    }
}

void test()
{
    Base *pbase1 = &derived;
    pbase1->func1();
    pbase1->func2();
}
```

```
class Base1
{
public:
    virtual void print1()
    {

    }
private:
    int _ibase1;
}


class Base2
{
public:
    virtual void print2()
    {

    }
private:
    int _ibase2;
}

class Base3
{
public:
    virtual void print3()
    {

    }
private:
    int _ibase3;
}


class Derived
:  virtual public Base1
,virtual public Base2
,virtual public Base3
{
public:
    virtual void print()
    {
```

```

    }
private:
    int _idervied;
}

_vbptr
_idervied;

vfptr
_ibase;



//_vfptr
_vbptr
_idervied
_vfptr
_ibase1
_vfptr
_ibase2
_vfptr
_ibase3
```

```
class Singleton
{
private:
    class AutoRelease
    {

    };
private:
    static AutoRelease _ar;
};
```